Grammar Checking as a Token Classification Task

Cody Daniels
Brigham Young University
Department of Computer Science
codymd@byu.edu

Abstract

In this paper, I explore fine-tuning pre-trained language models to perform grammar checking as a token classification task. I create my own dataset by using a found parallel corpus of grammar corrections and annotating it with the ERRANT library. I also convert word-level annotations to token-level so they are compatible with each base model. I try performing both generic and specific error identification. All models are able to perform generic error identification with great accuracy, although specific error identification proves slightly more challenging. Finally, I deploy my best performing generic error identification model as an API.

1 Introduction

Grammar checking is a common natural language processing task that enables everyone to be more professional and accurate communicators. Grammar checking is also particularly useful for learners of second languages, especially when the feedback is specifically labeled with the type of error.

There are several different ways to architect a grammar checking model. In this paper, I will explore building a grammar checker as a token classification model. I will build models for both generic and specific error identification.

2 Methodology

2.1 Datasets

I found a small, clean dataset on HuggingFace (`dteran/grammar`) that contains some simple English sentences with one or two errors and their corrections. It also contained some Spanish data that I had to filter out with a language detection library. Oddly enough, it has been updated to only include English to Spanish translations since I have downloaded it, but I retain the old data. I doubled the size of the dataset by taking the corrected examples and creating entries with perfect examples for a more balanced dataset. There are 2,784 examples in total. 80% of the examples were used for the training set, and the remaining 20% of examples were used for the test set.

2.2 Error Annotation

I used the library ERRANT to annotate each of the erroneous examples. The annotator takes the erroneous sentence and its correction and returns the type of error that occurred for each set of differing characters. These error labels were converted into BIO-style labels and aligned with the tokenized text. There were 49 different class labels in total. I created a second version of the dataset that replaced each specific error type with a generic error label in BIO format resulting in 3 classes ("O", "B-ERROR", and "I-ERROR").

2.3 Base Models

I chose to fine-tune `bert-base-uncased`, `xlnet-base-cased`, and `albert-base-v2` due to their diverse tokenization styles. BERT and ALBERT are both uncased, but XLNet only comes in a cased version. They are also relatively small in size and easy to work with. Each model was fine-tuned twice: once on the generic error dataset and a second time on the specific error dataset. I used a batch size of 24 and trained for 18 epochs.

3 Results

The results from each of the fine-tuning experiments can be seen in Table 1 for the generic error identification results and Table 2 for the specific error identification results. Overall, the models are very similar to each other in performance. XLNet performs the best across all metrics for the generic error identification task, but not by much. The figures for the specific error identification task are much more mixed, although the BERT-based model may perhaps have an edge.

Base Model	Accuracy	F1 Macro	F1 Micro	Prec. Macro	Prec. Micro	Recall Macro	Recall Micro
BERT	0.9887	0.9103	0.9103	0.9448	0.9448	0.8782	0.8782
ALBERT	0.9891	0.9176	0.9176	0.9647	0.9647	0.8750	0.8750
XLNet	**0.9907**	**0.9282**	**0.9282**	**0.9686**	**0.9686**	**0.8910**	**0.8910**

Table 1: Performance of fine-tuned BERT, ALBERT, and XLNet on the generic error identification task. Bolded values represent the best scores for each metric.

Base Model	Accuracy	F1 Macro	F1 Micro	Prec. Macro	Prec. Micro	Recall Macro	Recall Micro
BERT	**0.9895**	0.7899	**0.9212**	0.8429	**0.9694**	**0.7572**	**0.8776**
ALBERT	0.9870	**0.7941**	0.9008	**0.8789**	0.9569	0.7518	0.8510
XLNet	0.9879	0.7899	0.9048	0.8321	0.9436	0.7769	0.8691

Table 2: Performance of fine-tuned BERT, ALBERT, and XLNet on the specific error identification task. Bolded values represent the best scores for each metric.

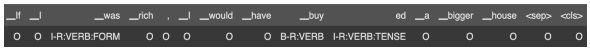

Figure 1: A misclassified example from the specific error identification task with the XLNet model.

Even with less than a few thousand examples, the models do very well on the generic error identification task with around a 0.92 F1 score. A slightly more accurate model may still be necessary to be truly production-ready, but it is good progress. On the other hand, all models are quite lacking on the specific error identification task. They score well on Micro metrics but lag behind in Macro metrics. This might reflect that each model is able to identify where errors are, but it has not quite figured out which type of error it is. That trend could be a result of a high number of error labels to choose from. Figure 1 shows an example of one such misclassification. XLNet predicted error labels in the right spots, but the type of error is just slightly off. It got the type of error correct (VERB) but predicted an I- where there should have been an B- and a TENSE subtype where there should have been FORM.

4 Deployment

I decided to deploy my XLNet model for the generic error identification task, since it performed the best overall. I used a Flask web server and a Redis queue to keep track of requests. Then, my XLNet model would periodically check the queue and perform inference in batches. See Figure 2 for an example of the system working. The left terminal is the model server, the middle terminal is the web server, and the right terminal sent a POST request with the sentence "I needs to use the bathroom." The model correctly returned that the verb "needs" is an error.

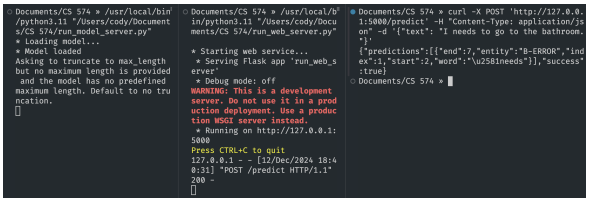

Figure 2: My deployed model in action.

5 Conclusion

The results of my simple grammar checker are promising, especially with such little data. Its success relies heavily on the rule-based annotator ERRANT to provide the training data, but after training it can provide error annotations without references. The specific error identification task proves a bit more difficult, but it would likely improve easily with more data. Deploying my model was a great learning experience.

5.1 Future Work

In the future, I would like to experiment with building a grammar checker for a different language like Finnish. I am unaware if there are any Finnish grammar annotators available, so finding data may be tricky. As a Finnish-as-a-second-language learner, a grammar checker with specific feedback would be invaluable for my studies.